PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

Temas Avanzados en Ingeniería de Software 1 – INF384
Laboratorio 3
(Segundo Semestre 2026)

Indicaciones generales:

- Duración 120 minutos.
- Puede consultar las diapositivas del curso, la documentación oficial de las herramientas y el repositorio base. No puede compartir código ni credenciales con otros alumnos. El uso de herramientas de IA generativa debe declararse y citarse según los lineamientos del curso.
- Los archivos indicados se subirán a Paideia, en la tarea del laboratorio, usando el nombre de archivo/s indicado en el enunciado.
- La presentación del trabajo influye en su calificación.

Puntaje: **20 puntos**

1. Objetivo

-Implementar validaciones de calidad y seguridad sobre la aplicación y publicar una imagen de contenedor, tomando en cuenta las validaciones.

-Implementar validaciones de sintaxis, integridad y seguridad sobre el código de infraestructura, validar plan y desplegar la infraestructura.

2. Pipeline de Aplicación VS Pipeline de Infraestructura

	Pipeline de aplicación	Pipeline de infraestructura
Archivo	`.github/workflows/app.yml`	`.github/workflows/infra.yml`
Se dispara con	cambios en `src/**`, `package*.json` y `Dockerfile`	cambios en `infra/**`
Es dueño de	la imagen docker que ejecuta la API (Lambda)	la existencia y la configuración de la API (Lambda)
Produce	Una imagen docker publicada en el registro	infraestructura aplicada

En el repositorio de Infraestructura, la función Lambda declara `ignore_changes = [image_uri]`, esto permite que el terraform no valide los cambios de esta URL dado que la aplicación la irá cambiando conforme libere nuevas versiones.

3. Alcance

Dentro del alcance

- Implementar buenas prácticas en el Dockerfile del repositorio base.
- Cuatro gates en el pipeline de aplicación: calidad (SonarCloud), detección de secretos, análisis de composición de dependencias y análisis estático.
- Build, tag y push de la imagen al registro, con el digest como salida del job.
- Pipeline de infraestructura completo: `fmt`, `init`, `validate`, análisis de políticas, `plan`, compuerta sobre el plan y `apply`.
- Inyectar 2 fallas para validar controles.

Fuera del alcance

- El job de despliegue. Viene escrito en el repositorio base y no se modifica.
- Escaneo de seguridad de la imagen de contenedor.
- El repositorio de imágenes (ECR) y el bucket del estado. Los crea el workflow `setup-infra` y no se declaran en Terraform.

Repositorio base: https://github.com/josedelcastillo/INF384-lab3-base

4. Prerrequisitos

- Hacer un fork del repositorio base **a su cuenta personal**.
- Sesión del entorno académico de AWS iniciada.
- Cuenta de SonarCloud configurada (pueden usar la misma del lab 2, pero el proyecto es nuevo: este es otro repositorio).
- No hay nada que instalar.

Todo el laboratorio se ejecuta en los runners de GitHub, que ya traen Docker, Terraform, la CLI de AWS y Node. No es necesario construir imágenes ni desplegar infraestructura desde su máquina, sin embargo pueden usar el GitHub Codespace si lo desean.

> El fork debe ir a una **cuenta personal**, no a una organización. Una de las actions que van a usar es gratuita solo en cuentas personales y en una organización pide una licencia.

5. Preparación previa a la sesión

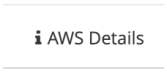
5.1 Credenciales

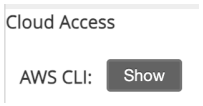
Las credenciales del entorno académico son temporales y caducan, deberás actualizarlas al iniciar el laboratorio.

1. Iniciar la sesión del laboratorio y copiar los tres valores: identificador de llave, llave secreta y token de sesión.
2. Cargarlos como secretos del repositorio con los nombres `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY` y `AWS_SESSION_TOKEN`. (RECUERDA hacerlo como secreto, si no expones tu ambiente al mundo)
3. Ejecutar el pipeline `verificar-credenciales`. Debe terminar en verde antes de continuar.

¿Cómo obtener los valores?

- Ingresa a tu laboratorio de AWS e inícialo.

- Ingresa a “AWS Details” 

- Dar click en SHOW para que aparezcan las credenciales. 

Para validar si su token está vigente y operativo pueden ejecutar manualmente el pipeline “`verificar-credenciales`”. Tener en cuenta que las credenciales duran 4 horas apróximadamente.

Cuando algo empiece a fallar sin que hayas cambiado nada, lo primero es volver a ejecutar `verificar-credenciales`. Si sale rojo, reinicia tu laboratorio AWS y actualiza las credenciales de AWS en los secrets de Github.

5.2 Creación inicial de la infraestructura

Una función Lambda de tipo imagen exige que la imagen del docker exista en el registry (ECR de AWS) antes de crear la función. Por eso `setup-infra` deja listo todo lo que el pipeline de infraestructura necesita para poder correr:

1. El bucket de S3 donde vive el archivo de estado, con versionado, acceso público bloqueado y cifrado.
2. El repositorio de imágenes (ECR).
3. Una imagen mínima inicial, con el tag `bootstrap`.

Pasos:

1. Ejecutar manualmente el workflow `setup-infra`, pasándole como parámetro el nombre del bucket: `inf384-tfstate-<su-codigo-pucp>`.
2. Al terminar, el resumen de la ejecución imprime el contenido exacto de `infra/backend.hcl`. Copiarlo en ese archivo y **confirmar el cambio en la rama principal**.

`setup-infra` es idempotente. Si falla a mitad de camino, volver a ejecutarlo.

El archivo “`infra/backend.hcl`” se versiona: el pipeline de infraestructura lo lee en el runner y fallará si no lo encuentra.

Ese primer commit dispara el pipeline de infraestructura, creando: el grupo de logs, la política del repositorio de imágenes y la función Lambda.

Para simplificar, el lock del estado de Terraform vive en el mismo bucket y no en DynamoDB como vimos en clase. No hay que hacer nada adicional para que funcione.

6. Explicación de repositorio base

Lo que trae

- `src/`: función Node con un handler que devuelve la versión, y sus pruebas unitarias.
- `package.json` y `package-lock.json`.
- `Dockerfile`: versión con malas prácticas.

- `sonar-project.properties`: configuración del análisis de calidad, ya resuelta. No hay que editarla.
- `infra/`: código Terraform de la función Lambda y los componentes que necesita para sus logs.
- `.github/workflows/app.yml`: pipeline para validar y desplegar la aplicación. Se debe completar con los pasos requeridos, salvo el job `desplegar` que viene completo.
- `.github/workflows/infra.yml`: pipeline para validar y desplegar infraestructura. Se debe completar.
- Auxiliares que no necesitan modificación, en `.github/workflows/`:
    - `verificar-credenciales.yml`: comprueba que las credenciales de AWS estén cargadas y que la sesión siga activa.
    - `setup-infra.yml`: crea el bucket del estado, el repositorio ECR y publica la primera imagen que el Lambda necesita para poder crearse.
    - `construir-imagen.yml`: construye la imagen, reporta el tamaño de cada etapa y corre las comprobaciones del criterio de aceptación de A1, sin publicar nada.

Lo que NO trae, y hay que agregar

- `.dockerignore`. Solo viene `.dockerignore.ejemplo`, con la explicación.
- `infra/backend.hcl`. Solo viene `infra/backend.hcl.ejemplo`.

7. Parte A · Pipeline de aplicación

A1 · Corregir el Dockerfile

Reescribir `Dockerfile` corrigiendo los cinco hallazgos marcados en el archivo. La imagen final debe cumplir:

- Imagen base de Lambda para Node con versión fija.
- Manifiesto y lock file copiados e instalados antes del código de la aplicación.
- Instalación desde el lock file.
- Construcción en dos etapas, donde la etapa final recibe únicamente el artefacto empaquetado.
- Sin valores de credencial declarados.
- Sin gestor de paquetes del sistema ni herramientas de depuración en la etapa final.

Agregar `.dockerignore`.

El repositorio incluye `esbuild` como dependencia de desarrollo y el script `npm run build`, que empaqueta el handler y sus dependencias en `dist/handler.js`. La imagen final debe ejecutar ese archivo empaquetado, sin el árbol de `node_modules`.

Sobre `USER` y `HEALTHCHECK` en este modelo de empaquetado: no será necesario declararlos ya que Lambda internamente lo maneja.

Para iterar sin esperar el pipeline completo, usar el workflow `construir-imagen`: construye, reporta el tamaño de cada etapa y no publica nada.

Criterio de aceptación: `docker history` sobre la imagen construida no muestra ningún valor de credencial, y la etapa final no contiene `node_modules` ni instalaciones con el gestor de

paquetes del sistema. La salida del workflow `construir-imagen` incluye las dos comprobaciones.

> El binario `dnf` viene dentro de la imagen base de Lambda y no se puede quitar de ahí. Lo que se evalúa es que la etapa final no lo **invoque**.

A2 · Compuertas de calidad y seguridad

Agregar cuatro jobs a `app.yml`. Los cuatro se ejecutan en paralelo y ninguno depende de otro.

Job	Qué evalúa	Condición de detención
`calidad`	Cobertura y reglas sobre el código nuevo	Solo informativo, no bloquea.
`secretos`	Credenciales en el código y en el historial	Cualquier hallazgo detiene
`dependencias`	Árbol completo del lock file contra vulnerabilidades conocidas	Solo informativo, no bloquea.
`estatico`	Patrones explotables en el código fuente	Solo informativo, no bloquea.

Requisitos:

- El job `secretos` usa `fetch-depth: 0`.
- El job `dependencias` se ejecuta sobre `package-lock.json`, con el árbol completo y no solo las dependencias de producción.
- El job `calidad` consume el reporte de cobertura que producen las pruebas, y necesita el historial completo del repositorio para distinguir el código nuevo.
- Que una compuerta no bloquee no significa que pueda quedar muda. El hallazgo tiene que verse en la ejecución sin abrir el log.

Configuración de SonarCloud. El job `calidad` necesita tres valores cargados en `Settings > Secrets and variables > Actions`:

Valor	Tipo
`SONAR_TOKEN`	Secret
`SONAR_ORG`	Variable
`SONAR_PROJECT_KEY`	Variable

El proyecto se crea **importando este repositorio** desde GitHub, igual que en el Laboratorio 2.

Es un proyecto nuevo: el del Laboratorio 2 corresponde a otro repositorio. El análisis ya viene configurado en `sonar-project.properties`.

Herramientas disponibles. No hace falta instalar nada en el runner. Estas son las referencias que pueden usar. Elegir otra es válido: se califica el gate configurado, sin importar el action usado..

Compuerta	Referencia	Observación
calidad	`SonarSource/sonarqube-scan-action@v8`	La misma del Laboratorio 2. Necesita `SONAR_TOKEN` y `SONAR_HOST_URL`
secretos	`gitleaks/gitleaks-action@v3`	Gratis solo en cuentas personales.
dependencias	`aquasecurity/trivy-action@v0.36.0`	No tiene tag flotante: hay que fijar la versión completa, `@v0.36.0` y no `@v0`
estatico	No existe action mantenida, se debe usar la imagen docker.	`Ver comando a continuación`

Ejemplo de step con invocación a docker de semgrep:

```
- name: Analizar el codigo fuente
  run: |
    docker run --rm \
      --volume "${PWD}:/src" \
      --workdir /src \
      --env SEMGREP_SEND_METRICS=off \
      semgrep/semgrep \
      semgrep scan \
        --config=p/default \
        --config=p/javascript \
        --json --quiet \
        --output=semgrep.json \
        src/
```

A3 · Build y publicación de la imagen

Agregar el job `publicar`, que depende de los cuatro anteriores. La imagen se publica con 2 tags independientes **sobre el mismo manifiesto**:

Tag	Naturaleza	Rol
`9a1c4f2`	inmutable, nace del commit	con este se publica; trazabilidad automática al código fuente
`prd`	móvil, se reasigna en cada promoción	qué versión está designada para ese ambiente

- `sha`: los siete primeros caracteres del commit. Es el tag con el que se construye y se publica la imagen, y del que se lee el digest. Nace del commit, así que siempre existe.
- `ambiente`: `prd` para la rama principal.

El despliegue del lambda va siempre con el digest de la imagen docker, que lo devuelve el push de la imagen docker al registry

El job debe además:

- Construir la imagen docker con `--platform linux/amd64`.
- Declarar el label `org.opencontainers.image.title` con el nombre de la aplicación.
- Ejecutarse solo desde la rama principal.
- Exponer el digest de la imagen publicada como salida del job, con el nombre `digest`, en formato `<registry>/<aplicacion>@sha256:....`

  El job `desplegar` consume esa salida. Si la nombran distinto, o si exponen solo el sha del digest sin el registro y la aplicación, el despliegue falla.

8. Parte B · Pipeline de infraestructura

B1 · Validación de sintaxis e integridad

Job `validar`, dos comprobaciones:

```
terraform init -backend-config=backend.hcl
terraform validate
```

Condición de detención: cualquiera falla.

`init` habla con el backend remoto, que vive en S3, así que este job necesita las credenciales de AWS.

B2 · Análisis de políticas

Compuerta	Referencia	Observación
`política IaC`	`aquasecurity/trivy-action@v0.36.0`	Solo escanear carpeta infra, fallar por HIGH y CRITICAL

Job `politicas`, sobre el código declarado:

- Ejecutar el análisis de configuración e infraestructura como código.
- Declarar en el workflow a partir de qué severidad un hallazgo detiene el pipeline.

A diferencia de las compuertas de la Parte A, **esta detiene**.

B3 · Plan y compuerta sobre el plan

Job `plan`, que depende de los dos anteriores:

```
terraform plan -out=tfplan
terraform show -json tfplan > tfplan.json
```

Regla · El plan se publica en el pull request, en texto legible, para que quien revisa vea la diferencia exacta antes de aprobar. Esta regla no detiene, informa.

B4 · Aplicación

Job `aplicar`, que depende de `plan`:

- Se ejecuta solo desde la rama principal.
- Aplica el plan guardado en el paso anterior, **sin recalcularlo**.

Cambio a aplicar en la sesión: subir la retención del grupo de logs de 1 a 7 días y la memoria de la función de 512 a 1024 MB.

9. Inyecciones de falla obligatorias

Inyección 1 · Gate de aplicación. Introducir en el código una credencial escrita. Demostrar que el pipeline se detuvo, en qué job, y que la imagen no se publicó al registro.

> Usar una credencial con forma realista. Las llaves de ejemplo de la documentación de AWS (`AKIAIOSFODNN7EXAMPLE` y su secreto) están en la lista de exclusión de las herramientas de detección: no producen ningún hallazgo y van a concluir que su compuerta no funciona.

Inyección 2 · Gate del plan terraform. Modificar el código de infraestructura de forma que el

10. Entregable

Subir a Paideia un único archivo PDF con el nombre `LAB3_<codigo-pucp>.pdf`, que contenga:

- Enlace al repositorio
- Pantallazo de la pantalla de commits.

11. Rúbrica

Criterio	Puntos
Dockerfile corregido: versión fija, dos etapas, lock file antes del código, sin credenciales, sin herramientas en la etapa final	3
Gate en el pipeline de aplicación con los 4 controles, con la condición de detención correcta para secretos.	6
Imagen publicada con los tags independientes sobre el mismo manifiesto, con label de aplicación, solo desde la rama principal, y el digest expuesto como salida	3
Pipeline de infraestructura: sintaxis, integridad y análisis de políticas con detención	3
Gate sobre el plan: detiene ante destrucción no autorizada, publica el plan en el pull request	4
Evidencia de las dos inyecciones de falla	1
Total	**20**

13. Errores frecuentes

Síntoma	Causa habitual	Dónde mirar
Pipeline de Setup-infra falla	La sesión del entorno académico se cerró. `sts get-caller-identity` sigue respondiendo, así que no se nota	Ejecutar `verificar-credenciales` `Actualizar credenciales en lab AWS.`
`403 Forbidden` de S3 sobre `terraform.tfstate`	Lo mismo de arriba. No es un problema del backend: sin permiso para listar, S3 responde 403 en vez de 404	Ejecutar `verificar-credenciales` `Actualizar credenciales en lab AWS.`
Error de autenticación en cualquier job de AWS	Token de sesión ausente o credenciales caducadas	Ejecutar `verificar-credenciales`
`terraform init` falla	Bucket de estado inexistente, o `backend.hcl` sin crear o mal declarado	Salida del workflow `setup-infra` y archivo `backend.hcl` en infra/
El job `calidad` falla aunque sea informativo	Falta `SONAR_TOKEN` o alguna de las dos variables. El job falla.	Secretos y variables del repositorio